

WEB-002 — Fondations — Mandat d'exécution (Claude Code)

World Skills Protocol · Chantier Web · Sprint WEB-002 Statut WEB-001B : v0.2 validé (WEB-001C).

Base : dépôt réel audité `erwanwts/opus-x` . Langue de travail : FR · Vocabulaire canonique EN préservé · `next-intl` (API Next 15 vérifiée).

Cadre. Exécution **locale uniquement**. Aucun déploiement, aucun service externe, aucun secret nouveau, aucun formulaire, aucune analytics dans ce sprint → **non concerné par le blocage SEC-02** (WEB-D4). **Interdiction** de déployer sur staging/Vercel tant que SEC-02 n'est pas confirmée. Règle : « *Nothing that has been proven may be broken by what comes next.* » Chaque lot ferme sur `npm run build vert + tests verts`. Aucune décision structurante silencieuse : tout écart → STOP.

Ordre des lots : **A** → **B** → **C** → **D**. A et B débloquent `[locale]` ; C pose le multilingue ; D pose le SEO/GEO technique de base.

LOT A (WEB-002.0) — Migration du Passport HTML public `/handle` → `/p/handle`

But. Libérer la racine dynamique pour `[locale]` (collision confirmée WEB-001A), **sans aucune régression** de confidentialité. **Aucun changement DB / RLS / API** — c'est un déplacement de route Next.js.

Opérations (exactes). 1. `git mv app/(public-passport)/[handle] app/(public-passport)/p` → la page vit désormais à `app/(public-passport)/p/[handle]/page.tsx` (URL `/p/{handle}`). 2. Vérifier qu'il ne reste **aucun** segment dynamique à la racine : `find app -maxdepth 2 -name "[[*]]" -type d` ne doit lister que des chemins **préfixés** (`p/[handle]` , `passports/[handle]` , `frameworks/[id]`) — jamais un `[x]` directement sous `app/` . 3. La page conserve **verbatim** : `createPublicClient` (`anon` , `persistSession:false`), la policy RLS `visibility='public'` comme seule barrière, la construction par **whitelist explicite**, le **404 non-énumérant** (aucune distinction privé/inexistant). **Ne rien réécrire d'autre**. 4. `app/passports/[handle]/route.ts` (API JSON) : **inchangé**. 5. **Aucune** redirection depuis `/handle` (jamais mise en circulation) ; **ne pas** recréer cette route via middleware.

Tests (repointés, verts avant fermeture). - `tests/integration/public-passport.integration.test.ts` : remplacer les URLs `/${handle}` par `/p/${handle}` . Conserver les assertions : 404 non-énumérant pour un handle inexistant **et** pour un handle privé (réponses **indistinguables**) ; un Passport `public` expose **uniquement** les champs whitelists. - Ajouter un test : une requête sur l'ancien chemin racine `/${handle}` **ne résout plus** (404), confirmant la libération de la racine.

Sortie A. `npm run build vert + tests confidentialité verts + racine dynamique libérée`. **Preuve exigée** : sortie `find` (§A.2) + run des tests public-passport.

LOT B (WEB-002.0-bis) — Root layouts multiples (`<html lang>` résolu, §10.1)

But. Permettre à `<html lang={locale}>` d'être **statique** côté site, sans casser le rendu prouvé de l'app. Pattern Next.js **multiple root layouts** via deux route groups — aucune URL applicative ne change (les route groups sont invisibles dans l'URL).

B.1 — Extraire les polices dans un module partagé. `lib/site/fonts.ts` :

```
// PRODUCT-001 §6.1 – trois voix. Module partagé par les deux root layouts.
import { Source_Serif_4, Inter, IBM_Plex_Mono } from 'next/font/google';

export const sourceSerif = Source_Serif_4({
  subsets: ['latin'], weight: ['400', '500', '600'],
  variable: '--font-source-serif', display: 'swap',
});
export const inter = Inter({
  subsets: ['latin'], weight: ['400', '500', '600'],
  variable: '--font-inter', display: 'swap',
});
export const plexMono = IBM_Plex_Mono({
  subsets: ['latin'], weight: ['400', '500'],
  variable: '--font-plex-mono', display: 'swap',
});
export const fontVars = `${sourceSerif.variable} ${inter.variable} ${plexMono.variable}`;
```

B.2 — Supprimer le root layout unique. `git rm app/layout.tsx`. (Il sera remplacé par deux root layouts de branche.) Conserver `import './globals.css'` — il sera importé par les deux nouveaux root layouts.

B.3 — Regrouper l'application sous (application) (URLs inchangées) :

```
git mv "app/(auth)" "app/(application)/(auth)"
git mv "app/(ceremony)" "app/(application)/(ceremony)"
git mv "app/(app)" "app/(application)/(app)"
git mv "app/(public-passport)" "app/(application)/(public-passport)"
```

Créer le **root layout applicatif** `app/(application)/layout.tsx` (comportement actuel préservé : `lang="fr"`, trois polices) :

```
import type { Metadata } from 'next';
import { fontVars } from '@lib/site/fonts';
import '@app/globals.css';

export const metadata: Metadata = {
  title: 'Opus X',
  description: 'Votre identité professionnelle, établie pour la vie.',
};

export default function ApplicationRootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="fr" className={fontVars}>
      <body>{children}</body>
    </html>
  );
}
```

(marketing)/page.tsx (racine /) : la racine est désormais gérée par le middleware (redirection 307, Lot C/ §10.3). Déplacer le **design** de cette landing vers (site)/[locale]/page.tsx (Lot C.7), puis `git rm "app/(marketing)/page.tsx"`. Ne pas laisser deux propriétaires de /.

B.4 — Root layout du site : créé au Lot C.6 (il dépend de `next-intl`).

Sortie B. `npm run build` vert · l'app rend toujours `<html lang="fr">` avec les 3 polices · toutes les URL applicatives (`/dashboard`, `/emission`, `/establish`, `/verify-email`, `/passport`, `/link`, `/me`, `/p/{handle}`, `/passports/{handle}`, `/api/*`, `/auth/callback`) résolvent comme avant. **Test de non-régression :** `tests/integration/guard.integration.test.ts` toujours vert (protection app intacte).

LOT C — Socle `next-intl` (en / fr / es, anglais canonique, fallback strict)

C.1 — Installer. `npm i next-intl` (aucun secret, local).

C.2 — Routing `i18n/routing.ts` :

```
import { defineRouting } from 'next-intl/routing';

export const routing = defineRouting({
  locales: ['en', 'fr', 'es'],
  defaultLocale: 'en',           // langue canonique institutionnelle ($?, x-default → /en)
  localePrefix: 'always',       // chaque langue a SON URL : /en /fr /es
  localeCookie: { name: 'NEXT_LOCALE' },
});

export type AppLocale = (typeof routing.locales)[number];
```

C.3 — Navigation `i18n/navigation.ts` :

```
import { createNavigation } from 'next-intl/navigation';
import { routing } from './routing';

export const { Link, redirect, usePathname, useRouter, getPathname } =
  createNavigation(routing);
```

C.4 — Request config `i18n/request.ts` :

```
import { getRequestConfig } from 'next-intl/server';
import { hasLocale } from 'next-intl';
import { routing } from './routing';

export default getRequestConfig(async ({ requestLocale }) => {
  const requested = await requestLocale;
  const locale = hasLocale(routing.locales, requested) ? requested : routing.defaultLocale;
  return {
    locale,
    messages: (await import(`../messages/${locale}.json`)).default,
  };
});
```

C.5 — Plugin `next.config.ts` (préserver `reactStrictMode`) :

```
import type { NextConfig } from 'next';
import createNextIntlPlugin from 'next-intl/plugin';

const nextConfig: NextConfig = { reactStrictMode: true };
const withNextIntl = createNextIntlPlugin();
export default withNextIntl(nextConfig);
```

C.6 — Root layout du site `app/(site)/[locale]/layout.tsx` (rend `<html lang={locale}>` , statique) :

```
import { NextIntlClientProvider, hasLocale } from 'next-intl';
import { setRequestLocale } from 'next-intl/server'; // (ancien nom : unstable_setRequestLocale)
import { notFound } from 'next/navigation';
import { routing } from '@/i18n/routing';
import { fontVars } from '@/lib/site/fonts';
import '@app/globals.css';

export function generateStaticParams() {
  return routing.locales.map((locale) => ({ locale }));
}

type Props = { children: React.ReactNode; params: Promise<{ locale: string }> };

export default async function SiteRootLayout({ children, params }: Props) {
  const { locale } = await params;
  if (!hasLocale(routing.locales, locale)) notFound();
  setRequestLocale(locale); // APRÈS le check, AVANT toute API next-intl → rendu statique
  return (
    <html lang={locale} className={fontVars}>
      <body>
        <NextIntlClientProvider>{children}</NextIntlClientProvider>
      </body>
    </html>
  );
}
```

Règle de rendu statique : `setRequestLocale(locale)` doit être appelé **en tête de chaque page ET layout** du site avant toute API `next-intl`, sinon Next.js bascule en rendu dynamique (perte SSG/cache — §10.1).

C.7 — Home localisée `app/(site)/[locale]/page.tsx` — **portage du design de la landing existante** (registre éditorial : navy dominant, **or absent**, un seul CTA institutionnel → `/establish`). Le texte vient des messages, le TITRE de l'écran d'entrée reste en anglais (cas frontière `passport.strings.ts`). Squelette :

```
import { setRequestLocale } from 'next-intl/server';
import { useTranslations } from 'next-intl';
import { Link } from '@/i18n/navigation';

export function generateStaticParams() {
  return [{ locale: 'en' }, { locale: 'fr' }, { locale: 'es' }];
}

export default function HomePage({ params }: { params: { locale: string } }) {
  setRequestLocale(params.locale);
  return <HomeContent />;
}

function HomeContent() {
  const t = useTranslations('landing');
  // Reprendre EXACTEMENT la structure de app/(marketing)/page.tsx (hero, signaux, footer),
  // en remplaçant fr.landing.* par t('...'). CTA unique → /establish (hors locale, route app).
  return (/* ... design porté depuis la landing ... */ null);
}
```

Le lien `/establish` pointe vers la route **applicative** (hors `[locale]`) — c'est voulu : l'app exécute, le site oriente.

C.8 — Messages `messages/en.json`, `messages/fr.json`, `messages/es.json`. Seed initial (à compléter au fil des pages ; `fr.json` réutilise `lib/i18n/fr.ts` comme source). Exemple `fr.json` :

```
{
  "landing": {
    "headline": "Votre identité professionnelle, établie pour la vie.",
    "subhead": "...",
    "message": "On n'ouvre pas un compte. On établit une identité professionnelle.",
    "cta": "Continuer",
    "signals": { "...": "..." },
    "footer": "..."
  }
}
```

Fallback strict (§10.2) : une clé n'est publiée dans une locale **que** si elle est réellement traduite et validée. Une page dont le contenu n'existe pas dans `fr / es` **n'est pas générée** pour cette locale (voir C.9), n'apparaît ni au sitemap ni aux `hreflang` ; l'UI peut proposer explicitement `/en/...` . **Jamais** d'anglais servi sous `/fr` ou `/es` .

C.9 — Fallback strict, mécanique. Pour toute route à contenu (docs, glossary, faq, use-cases...), `generateStaticParams` **itère le contenu réellement présent par locale** (pas le produit cartésien `locales × slugs`). Helper `lib/site/content.ts` :

```
// Retourne uniquement les couples (locale, slug) dont la traduction existe et est publiée.
export function publishedParams(type: string) {
  // lit content/{type}/{locale}/*.mdx → ne renvoie que les entrées status:published + classification PUBLIC*
  // → une locale sans traduction ne produit AUCUN param (404 honnête, hors sitemap/hreflang).
}
```

C.10 — Middleware composé `middleware.ts` — **le point sensible**. `next-intl` gère la locale (et la **redirection 307 de /** selon cookie → `Accept-Language` → `en`, ce qui **réalise WEB-D5**) ; la **session Supabase** ne s'exécute que sur les routes app protégées. Segments **réservés** (non localisés) = `handlers/pages` existants.

```

import createMiddleware from 'next-intl/middleware';
import { createServerClient, type CookieOptions } from '@supabase/ssr';
import { NextResponse, type NextRequest } from 'next/server';
import { routing } from '@/i18n/routing';

const intlMiddleware = createMiddleware(routing);

// Premiers segments RÉSERVÉS à l'application / aux handlers (jamais localisés).
const RESERVED = new Set([
  'dashboard', 'emission', 'establish', 'verify-email', 'passport', 'link',
  'me', 'p', 'passports', 'auth', 'api', 'frameworks', 'issuers',
]);
// Sous-ensemble RÉELLEMENT protégé (garde de session – logique existante préservée).
const isProtected = (path: string) =>
  path.startsWith('/dashboard') || path.startsWith('/emission') ||
  path === '/passport' || path.startsWith('/passport/');

async function appSession(request: NextRequest) {
  let response = NextResponse.next({ request });
  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll: () => request.cookies.getAll(),
        setAll: (toSet: { name: string; value: string; options: CookieOptions }[]) => {
          toSet.forEach(({ name, value }) => request.cookies.set(name, value));
          response = NextResponse.next({ request });
          toSet.forEach(({ name, value, options }) => response.cookies.set(name, value, op-
tions));
        },
      },
    }
  );
  const { data: { user } } = await supabase.auth.getUser();
  if (isProtected(request.nextUrl.pathname) && !user)
    return NextResponse.redirect(new URL('/establish', request.url));
  if (isProtected(request.nextUrl.pathname) && user && !user.email_confirmed_at)
    return NextResponse.redirect(new URL('/verify-email', request.url));
  return response;
}

export async function middleware(request: NextRequest) {
  const first = request.nextUrl.pathname.split('/')[1] ?? '';
  // Déjà localisé (/en /fr /es) → next-intl.
  if (hasLocalePrefix(first)) return intlMiddleware(request);
  // Segment réservé app/handler → session (si protégé) sinon pass-through, JAMAIS de locale.
  if (RESERVED.has(first)) {
    return isProtected(request.nextUrl.pathname) ? appSession(request) : NextResponse.next();
  }
  // Chemin de site non préfixé (/ , /about, ...) → next-intl ajoute la locale (307, WEB-D5).
  return intlMiddleware(request);
}

function hasLocalePrefix(seg: string) {
  return (routing.locales as readonly string[]).includes(seg);
}

export const config = {
  // Exclut assets, _next et fichiers à extension ; branche à l'intérieur.
  matcher: ['/(?!_next/static|_next/image|favicon.ico|.*\\..*).*/'],
};

```

Impact perf/cache : les pages [locale] ne lisent **jamais** headers/cookies dans leur rendu (locale = param + setRequestLocale) → **statiques/CDN-friendly**. `getUser()` ne s'exécute **que** sur `/dashboard|/emission|/passport` . `/` ne rend aucun corps (307).

Sortie C. `npm run build` vert . `/` → 307 vers `/en|/fr|/es` (cookie → Accept-Language → en) . `/en` rend la Home statiquement . une locale sans traduction → **404 honnête**, absente du sitemap/hreflang . app intacte (guard vert).

LOT D — Fondation SEO/GEO technique

D.1 — `metadataBase` + `defaults` dans le root layout site (ou `lib/site/metadata.ts`) : `metadataBase`: new URL('https://opusx.world'), titre template, defaults OG/Twitter. **generateMetadata par page** avec `alternates.canonical` + `alternates.languages` **limités aux traductions réelles** (§10.2).

D.2 — `app/sitemap.ts` (alternates linguistiques réels uniquement) :

```
import type { MetadataRoute } from 'next';
import { routing } from '@/i18n/routing';

const BASE = 'https://opusx.world';

export default function sitemap(): MetadataRoute.Sitemap {
  // Pour chaque entrée de contenu PUBLIÉE, n'émettre que les locales réellement traduites,
  // et déclarer languages.alternates uniquement pour celles-ci (jamais de /fr fantôme).
  const staticEntries = ['', 'passport', 'wsp', 'issuers', 'organizations',
    'developers', 'about', 'contact', 'glossary', 'faq', 'developers'];
  // ... construire { url: `${BASE}/${locale}/${path}`, alternates: { languages: {...réelles...} } }
  return [/* ... */];
}
```

D.3 — `app/robots.ts` :

```
import type { MetadataRoute } from 'next';
export default function robots(): MetadataRoute.Robots {
  return {
    rules: [{ userAgent: '*', allow: '/', disallow: ['/api/', '/me/'] }],
    sitemap: 'https://opusx.world/sitemap.xml',
    host: 'https://opusx.world',
  };
}
```

Les pages non publiées (legal sans texte humain, pricing) portent `robots: { index:false }` **par page** (WEB-D3), pas via `robots.txt` .

D.4 — `x-default` : dans les `alternates.languages` , ajouter la clé `'x-default'` → URL `/en` de la page. `hreflang` uniquement entre traductions existantes.

Sortie D. `sitemap.xml` valide (alternates réels) . `robots.txt` valide . métadonnées canoniques + hreflang cohérents . pages non publiées en `noindex` .

Conditions de sortie WEB-002 (Definition of Done du sprint)

1. `npm run build` **vert** (site + app).
2. Migration `/p/{handle}` faite ; **racine dynamique libérée** ; confidentialité **intacte** (404 non-énumérant).
3. `<html lang>` : `en/fr/es` côté site (statique), `fr` côté app — **sans régression** (guard + rendu app verts).
4. `/` → **307** cookie → `Accept-Language` → `en` ; `x-default` → `/en` .
5. **Fallback strict** prouvé : locale sans traduction → 404 honnête, hors sitemap/hreflang ; **jamais** d'anglais sous `/fr` · `/es` .
6. `sitemap.ts` / `robots.ts` valides ; metadata + hreflang réels.
7. Tests verts (confidentialité repointée, guard, non-régression app, absence de génération sans traduction).
8. **Aucun déploiement** (SEC-02) ; **aucun secret nouveau** ; travail 100 % local.
9. Preuves brutes fournies par lot (find, runs de tests, sortie build).

Reste hors périmètre WEB-002 (sprints suivants) : pages institutionnelles de contenu (WEB-003), documentation MDX (WEB-004), Knowledge Center (WEB-005), 20–30 pages GEO + blocs LLM (WEB-006). Ces sprints dépendent des décisions résiduelles §23 (classification `docs/` , textes juridiques, profil Commando) et, pour tout déploiement, de **SEC-02**.

WEB-002 — Fondations — Mandat d'exécution. Aucun document normatif modifié. Exécution locale, aucun déploiement avant SEC-02.